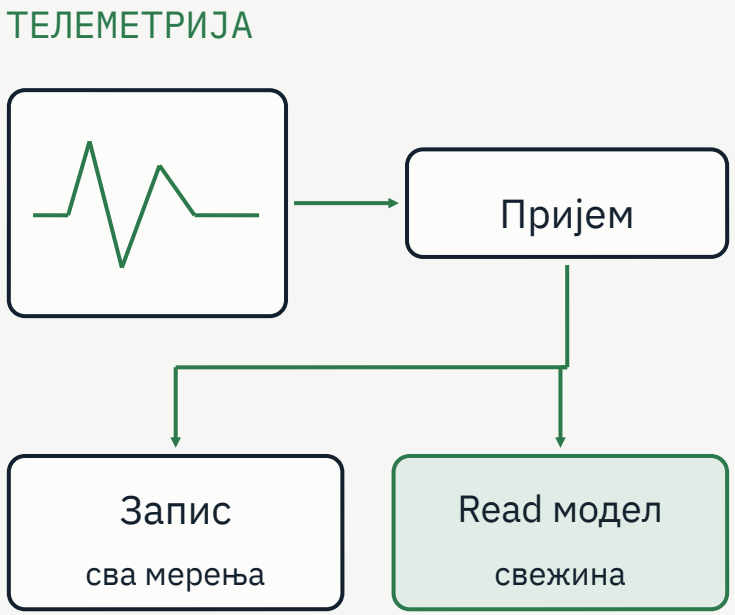
ПРИМЕЊЕНО СОФТВЕРСКО ИНЖЕЊЕРСТВО

ОСНОВЕ ДИСТРИБУИРАНОГ ПРОГРАМИРАЊА

Ток података и read модели

Од мрежне поруке до поуздане информације



Факултет техничких наука · Универзитет у Новом Саду

Нови Сад

ПРОВЕРА

Садржај презентације

01

Појмови и границе

Кључни појмови, одговорности компоненти и конкретан failure сценарио.

03–10

02

Дистрибуирани ток

Шема тока и гаранције које повезују компоненте система.

11–13

03

Код и failure тестови

Покретање примера, доменско правило, тест и видљиви исходи.

14–18

04

Примена и провера

Принципи, замке, задатак уз пример и примена на пројектну целину.

19–26

ПРВИ ДЕО

01

Појмови и границе

Кључни појмови, одговорности компоненти и конкретан failure сценарио.

01 · ПОЈМОВИ

Тема и проблем

Раздвојити пријем, валидацију, дистрибуцију и читање података са јасним границама одговорности.

Податак који је стигао са мреже још нема потврђено порекло, квалитет ни облик погодан за сваког корисника система.

НАПОМЕНА

Раздвојени слојеви чувају порекло информације и допуштају различите погледе на податке, уз отворено приказану цену кашњења.

01 · ПОЈМОВИ

Четири појма теме

Податак који стигне преко мреже још није поуздана пословна информација.

01 **Пријем**

извор и основна валидација

02 **Нормализација**

интерни облик и квалитет

03 **Дистрибуција**

независни конзументи

04 **Read модел**

поглед за коришћење

01 · ПОЈМОВИ

Пријем поруке

Пријем проверава извор, основну исправност и идентитет поруке пре него што податак постане део унутрашњег тока. Невалидна порука мора имати видљив исход.

01 Проверава извор и исправност пре уласка у систем.

02 Невалидна порука има видљив, не тих исход.

03 Прва линија одбране од лошег податка.

Нормализација

Нормализација претвара спољашњи формат у стабилан унутрашњи облик уз очувано порекло и време. Тако остатак система не зависи од детаља сваког извора.

01 Претвара спољашњи формат у стабилан унутрашњи облик.

02 Чува порекло и време настанка поруке.

03 Остатак система не зависи од детаља извора.

01 · ПОЈМОВИ

Усмеравање

Усмеравање шаље обрађену поруку само компонентама које заиста имају разлог да је тумаче. Оно одваја одговорности и ограничава непотребне зависности.

01 Шаље поруку само онима којима је заиста потребна.

02 Одваја одговорности између компоненти.

03 Ограничава непотребне зависности.

01 · ПОЈМОВИ

Read модел

Read модел је приказ прилагођен читању, а не нужно ауторитативно стање. Зато треба јасно показати када је последњи пут освежен и колико може каснити.

01 Приказ за читање, не ауторитативно стање.

02 Мора показати када је последњи пут освежен.

03 Застарелост је видљива, не скривена од корисника.

Пример: порука стиже два пута и касно

Телеметријска порука може стићи два пута, а њено време настанка може бити старије од времена пријема.

01 Пријем проверава извор и минимални облик поруке.

02 Нормализација чува квалитет и разлику између времена настанка и пријема.

03 Read модел отворено приказује свежину информације и не претвара дупликат у нови пословни догађај.

ДРУГИ ДЕО

02

Дистрибуирани ток

Шема тока и гаранције које повезују компоненте система.

02 · ТОК

Шема тока података

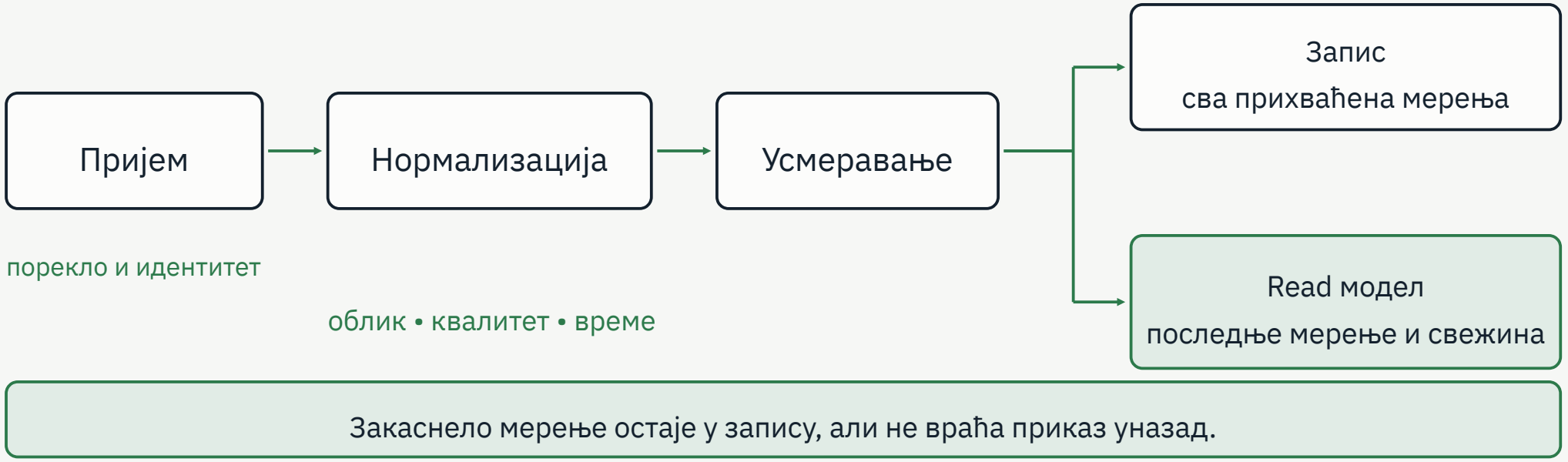
Податак постаје поуздан тек када су познати његово порекло, квалитет и границе приказа.

Елемент	Улога у току
Пријем	извор и идентитет поруке
Нормализација	квалитет и унутрашњи облик
Дистрибуција	независна обрада
Read модел	поглед са границом свежине

02 · ДИЈАГРАМ

Запис и приказ имају различиту сврху

Свако прихваћено мерење остаје у запису, док Read модел прати најновији sequence.



ТРЕЋИ ДЕО

03

Код и failure тестови

Покретање примера, доменско правило, тест и видљиви исходи.

Покретање примера

Пример раздваја запис од приказа.

terminal · Корен Git репозиторијума nastava.

```
cd "03 Osnove Distribuiranog Programiranja/04 Primeri/05 Tok podataka read model"
dotnet run --project src/Odp.Vezba05.ConsoleUi
dotnet test Odp.Vezba05.sln
```

Правило у коду

Запис чува сва прихваћена мерења, приказ чува само последње.

C# · доменско правило

```
public (StationView View, bool Changed) Apply(StationView? current, TelemetryReading reading)
{
    ArgumentNullException.ThrowIfNull(reading);
    if (current is not null && reading.Sequence <= current.LastSequence)
        return (current, false);
    return (
        new StationView(
            reading.StationId,
            reading.Sequence,
            reading.MeasuredAt,
            reading.SignalStrengthDbm),
        true);
}
```

03 · КОД И ТЕСТ

Тест који задаје квар

Мерење које закасни остаје у запису, али не мења приказ.

C# · NUnit

```
[Test]
public void Ingest_WhenOlderReadingArrivesLate_KeepsItInTheLogButNotInTheView()
{
    _handler.Ingest(Reading(3, -88));
    var outcome = _handler.Ingest(Reading(2, -97));
    using (Assert.EnterMultipleScope())
    {
        Assert.That(outcome, Is.EqualTo(new IngestOutcome(true, TelemetryCodes.Stored, ViewChanged:
            false)));
        Assert.That(_log.ForStation("GS-NOVI-SAD"), Has.Count.EqualTo(2));
        Assert.That(_views.Find("GS-NOVI-SAD")!.LastSequence, Is.EqualTo(3));
    }
}
```

Испис демонстрације

Приказ који дуго није освежен остаје видљив, али је означен као застарео.

console · исходи демонстрације

```
merenje #1 -> reading_stored, prikaz promenjen: True
merenje #3 -> reading_stored, prikaz promenjen: True
merenje #2 -> reading_stored, prikaz promenjen: False
merenje #0 -> sequence_must_be_positive, prikaz promenjen: False
[posle prijema] #3 -88 dBm, staro 5 s, zastarelo: False
[dva minuta bez novih merenja] #3 -88 dBm, staro 125 s, zastarelo: True
Zapis sadrži 3 merenja, prikaz samo poslednje.
```

ЧЕТВРТИ ДЕО

04

Примена и провера

Принципи, замке, задатак уз пример и примена на пројектну целину.

04 · ПРИМЕНА

Принципи и замке

Принцип важи тек када је препозната и замка која га тихо поништава.

ПРИНЦИПИ

- Стабилан идентитет поруке омогућава да се препозна њено понављање.
- Време настанка и време пријема имају различито значење.
- Read модел показује корисну слику, али мора знати границу своје свежине.

ЗАМКЕ

- Тихо одбацивање поруке без разлога и трага.
- Мешање времена извора и пријема.
- Read модел који дуплира пословни ефекат при поновљеној поруци.

04 · ПРИМЕНА

Типичне грешке

Ове грешке се најчешће понављају када се принцип површно примени.

01 Тихо одбацивање проблематичне поруке.

02 Мешање времена настанка и пријема.

03 Дупли пословни ефекат при поновљеној поруци.

04 · ПРИМЕНА

Како се принцип примењује

Кораци показују како се принцип препознаје и доследно спроводи у развоју.

01 Задржати идентитет и квалитет поруке кроз ток.

02 Одредити где се порука одбија.

03 Приказати границу свежине read модела.

04 · ПРИМЕНА

Питање за дискусију

Одговор се образлаже failure сценаријем и тестом, а не описом срећног пута.

Који податак код вас сме да касни и како би корисник требало да види да није потпуно свеж?

ЗА ВЕЖБУ

Који податак у вашем систему може каснити, а ипак остати користан? Како корисник треба да види ту ограду?

04 · ПРИМЕНА

Задатак уз пример

Проширење примера уз вежбу, правило иде у домен, а доказ у тест.

01 Додати use-case RebuildStationView који брише приказ станице и поново га изводи из записа.

02 Тестом показати да је резултат исти без обзира на редослед којим су мерења првобитно стигла.

Примена на пројекат

Принцип са вежбе примењује се на додељену пројектну целину.

01 Идентификовати конкретну дистрибуирану одлуку у својој целини.

02 Имплементирати је са стабилним кодом исхода.

03 Припремити поновљив failure сценарио и тест који га доказује.

04 · ПРИМЕНА

Резиме

Тим описује који слој сме да одбије поруку, који јој додељује квалитет и који је чува за касније читање.

Податак постаје поуздан тек када су познати његово порекло, квалитет и границе приказа.

НАПОМЕНА

Наредна вежба: Команде, послови и идемпотентност.